



DevSupportAI

Asistente inteligente de soporte técnico con RAG, datos estructurados y tool calling

Angular	TypeScript	Python	FastAPI
SQLite	ChromaDB	Groq	Sentence Transformers

Objetivo del proyecto

Construir un chatbot web de soporte capaz de decidir entre conocimiento general, documentación propia y datos estructurados, manteniendo una arquitectura clara entre frontend, API, recuperación de información y modelo de lenguaje.

Este documento describe únicamente la versión estable desarrollada. No incluye como funcionalidades actuales despliegue en producción, autenticación, historial persistente, pedidos, reservas ni la futura adaptación a un restaurante.

1. Arquitectura y stack tecnológico

DevSupportAI está dividido en un frontend Angular y un backend FastAPI. El navegador envía las consultas mediante HTTP; el backend coordina el modelo de lenguaje, el sistema RAG y las consultas a SQLite, y devuelve una respuesta estructurada al frontend.

Angular / TypeScript
Interfaz y contexto reciente



FastAPI / Python
API y orquestación



IA + datos
Groq · RAG · SQLite

Frontend

- Angular con componentes standalone y TypeScript.
- HttpClient para consumir la API y environment.ts para configurar su URL.
- Renderizado de respuestas Markdown con marked, manteniendo la sanitización de Angular.
- Widget flotante con apertura/minimización, indicador de escritura, scroll y visualización del origen de cada respuesta.

Backend y datos

- Python + FastAPI, modelos Pydantic, Uvicorn, CORS y configuración mediante python-dotenv.
- SQLite para recursos técnicos estructurados.
- ChromaDB persistente para el índice vectorial del sistema RAG.
- sentence-transformers/all-MiniLM-L6-v2 para generar embeddings localmente.

IA

- Groq como proveedor de inferencia mediante API compatible con OpenAI.
- Modelo configurado: openai/gpt-oss-20b.
- Generación de respuestas y tool/function calling real.

2. Funcionamiento de la IA

La petición no se limita a enviar una pregunta a un LLM. El backend dispone de distintas fuentes de información y devuelve al frontend el modo utilizado: rag, database o general.

1	El usuario escribe una consulta en el widget Angular.
2	Angular envía el mensaje y hasta 10 mensajes recientes de la sesión a POST /chat.
3	FastAPI prepara el contexto y las herramientas disponibles para el modelo.
4	Cuando corresponde, el modelo realiza una tool call para consultar SQLite; para conocimiento documental se recupera contexto mediante RAG.
5	El resultado recuperado se utiliza para construir la respuesta final.
6	Angular muestra la respuesta y su procedencia; en RAG también muestra los documentos consultados.

Tool calling y SQLite

El modelo puede solicitar la ejecución de herramientas definidas por el backend. DevSupportAI implementa `search_support_resources` para localizar recursos y `get_resource_details` para recuperar un recurso concreto por ID. FastAPI interpreta los argumentos JSON, ejecuta la función Python correspondiente y proporciona el resultado al modelo para generar la respuesta final.

Importante: no se trata de un enrutado basado únicamente en palabras clave. La llamada a herramientas se realiza mediante el mecanismo de tool calling del modelo.

Contexto conversacional

El frontend envía el historial reciente y el backend utiliza hasta los últimos 10 mensajes para mantener continuidad durante la sesión. El historial no se almacena de forma persistente.

3. Sistema RAG

El sistema RAG permite responder utilizando documentación propia en lugar de depender exclusivamente del conocimiento general del modelo.

Documentos Markdown	→	Fragmentación	→	Embeddings
Embeddings	→	ChromaDB	→	Recuperación
Contexto relevante	→	LLM	→	Respuesta

- La base de conocimiento actual contiene angular.md, fastapi.md y javascript.md.
- Los documentos se dividen en fragmentos de aproximadamente 500 caracteres, respetando párrafos.
- Los embeddings se generan con all-MiniLM-L6-v2 y se almacenan en una colección persistente de ChromaDB.
- La consulta se transforma en embedding, se recuperan candidatos y se calcula similitud coseno.
- Se utiliza un umbral de similitud de 0.45 y se devuelven como máximo 2 documentos relevantes.
- Un hash SHA-256 permite detectar cambios en la documentación y evitar reconstruir innecesariamente el índice.

Casos comprobados durante el desarrollo

Se verificaron consultas sobre CORS entre Angular y FastAPI, diferencias entre const y let en JavaScript, preguntas generales fuera de la documentación y consultas/follow-ups sobre recursos almacenados en SQLite. También se comprobó un follow-up de CORS con Angular en el puerto 4300 recuperando documentación relevante.

La interfaz hace visible esta arquitectura al usuario mediante etiquetas como Documentación consultada, Base de datos consultada y Conocimiento general.

4. Ingeniería, interfaz y calidad

API y robustez

- Modelos Pydantic para validar las estructuras de petición y respuesta.
- CORS limitado al origen del frontend configurado mediante variable de entorno.
- Tratamiento diferenciado de errores de autenticación, rate limit, timeout, conexión y errores de API del proveedor de IA.
- Las consultas SQLite filtran recursos activos; los fallos de base de datos se convierten en respuestas HTTP controladas.
- El RAG puede degradarse a una lista vacía si falla la recuperación, evitando exponer trazas al frontend.

Interfaz

- Widget flotante de soporte con identidad visual propia, favicon y landing de presentación.
- Apertura y minimización mediante el mismo botón, animaciones y diseño responsive.
- Indicador de escritura, control de scroll y aviso flotante cuando la respuesta llega fuera de la zona visible.
- Markdown para respuestas enriquecidas y estilos específicos para enlaces, código y fuentes.

Configuración y control de versiones

- La clave de Groq permanece en backend/.env y no se expone al frontend.
- .env.example documenta las variables necesarias sin incluir secretos.
- .gitignore excluye entorno virtual, base SQLite local, ChromaDB, estado del RAG y archivos de entorno.
- Proyecto versionado con Git/GitHub; la versión estable previa a futuras adaptaciones quedó marcada como v1.0-base.

Dependencias directas del backend

fastapi 0.136.1 · uvicorn 0.46.0 · openai 3.1.0 · python-dotenv 1.2.3 · chromadb 1.5.9 · sentence-transformers 6.0.0

5. Resultado y aprendizajes

DevSupportAI combina conceptos habituales de desarrollo web con una integración de IA que utiliza distintas fuentes de información. El proyecto demuestra la conexión entre una SPA en Angular y una API Python, además de recuperación vectorial y ejecución de herramientas desde un LLM.

Competencias demostradas

- Desarrollo full-stack: comunicación Angular - FastAPI mediante HTTP/JSON.
- Diseño de APIs: endpoints, validación de datos, CORS y gestión de errores.
- IA aplicada: integración de un LLM externo, contexto conversacional y tool calling.
- RAG: chunking, embeddings, almacenamiento vectorial, recuperación y selección por similitud.
- Persistencia: SQLite para datos estructurados y ChromaDB para vectores.
- Buenas prácticas: variables de entorno, separación de secretos, dependencias reproducibles y Git.

Alcance actual

Implementado: chatbot web, API, RAG, búsqueda vectorial, SQLite, tool calling, contexto temporal de conversación, identificación de fuentes/modos, tratamiento de errores e interfaz de portfolio.

No forma parte de esta versión: despliegue de producción, autenticación de usuarios, memoria persistente de conversaciones, pedidos, reservas o integración específica con un restaurante. Estas posibilidades se mantuvieron como evolución futura y no se presentan como funcionalidades desarrolladas.

DevSupportAI · Proyecto de portfolio orientado a desarrollo web e Inteligencia Artificial